



# FREELANCE HIRING PLATFORM



**2026**

Prepared by :

**Ali Hani Elborai**

**Mohamed Kamel**

**Antwan Micheal**

**221007553**

**221006482**

**221007568**

# Table of Contents

## Freelance Hiring Platform Project Report

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Project Description .....</b>                      | <b>1</b>  |
| <b>1. Introduction .....</b>                          | <b>1</b>  |
| <b>2. Project Objectives .....</b>                    | <b>1</b>  |
| 2.1 Creating a User-Friendly Freelance Platform ..... | 1         |
| 2.2 Supporting Job Seekers .....                      | 1         |
| 2.3 Supporting Employers .....                        | 2         |
| 2.4 Supporting Admin Users .....                      | 2         |
| 2.5 Using Modern Full-Stack Technologies .....        | 2         |
| 2.6 Organizing Data Using MongoDB .....               | 2         |
| <b>3. Technologies Used .....</b>                     | <b>3</b>  |
| 3.1 JavaScript .....                                  | 3         |
| 3.2 React and Vite .....                              | 3         |
| 3.3 CSS .....                                         | 3         |
| 3.4 Node.js and Express .....                         | 3         |
| 3.5 MongoDB and Mongoose .....                        | 4         |
| 3.6 JWT, bcrypt, Nodemailer, and Vercel .....         | 4         |
| <b>4. System Users and Roles .....</b>                | <b>4</b>  |
| 4.1 Job Seeker .....                                  | 4         |
| 4.2 Employer .....                                    | 5         |
| 4.3 Admin and Super Admin .....                       | 5         |
| <b>5. Main Features of the Website .....</b>          | <b>5</b>  |
| 5.1 Home Page and Layout .....                        | 5         |
| 5.2 Login and Authentication .....                    | 5         |
| 5.3 Create Job Page .....                             | 6         |
| 5.4 Job Browsing and Applications .....               | 6         |
| 5.5 Job Seeker Dashboard .....                        | 6         |
| 5.6 Job Seeker Profile Page .....                     | 6         |
| 5.7 Employer Dashboard .....                          | 6         |
| 5.8 Application Status and Email Notifications .....  | 6         |
| 5.9 Admin Panel .....                                 | 6         |
| <b>6. Database Design .....</b>                       | <b>7</b>  |
| 6.1 Main Database Collections .....                   | 7         |
| 6.1.1 Users Collection .....                          | 7         |
| 6.1.2 Jobs Collection .....                           | 7         |
| 6.1.3 Embedded Applications .....                     | 8         |
| 6.1.4 Admins Collection .....                         | 8         |
| 6.2 Importance of the Database .....                  | 8         |
| <b>7. Implementation Overview .....</b>               | <b>9</b>  |
| 7.1 Job Seeker Flow .....                             | 9         |
| 7.2 Employer Flow .....                               | 9         |
| 7.3 Admin Flow .....                                  | 9         |
| 7.4 Frontend Implementation .....                     | 9         |
| 7.5 Backend, API, and Deployment .....                | 10        |
| <b>8. Testing and Validation .....</b>                | <b>10</b> |
| 8.1 Job Seeker Testing .....                          | 10        |
| 8.2 Employer Testing .....                            | 10        |
| 8.3 Admin and Deployment Testing .....                | 10        |
| <b>9. Conclusion .....</b>                            | <b>10</b> |
| <b>10. Future Enhancements .....</b>                  | <b>11</b> |
| <b>Final Summary .....</b>                            | <b>11</b> |

## Project Description:

This project is a web-based freelance hiring platform designed to connect job seekers with employers. The updated system allows employers to post jobs, manage applicants, accept or reject applications, and notify applicants by email. Job seekers can browse jobs, apply using their contact details and CV link, create a profile, add skills, and track application status.

## 1. Introduction

The freelance market has grown rapidly because many people prefer flexible work opportunities and many companies need a fast way to find suitable workers for specific tasks or projects. Because of this, freelance hiring platforms have become important for organizing the hiring process digitally.

Our project is a **Freelance Hiring Platform** that connects three main types of users: **job seekers, employers, and admins**. Job seekers can search for available jobs, apply for suitable jobs, manage their profile, and track whether an application is in process, accepted, or rejected. Employers can create job posts, view jobs they added, review applicants, and update application decisions.

The platform was developed as a full-stack web application using **React** and **Vite** for the frontend, **CSS** for styling, **Node.js** and **Express** for backend APIs, and **MongoDB** with **Mongoose** for storing data. It also uses **JWT authentication**, password hashing, email notification utilities, and Vercel deployment configuration.

## 2. Project Objectives

The main objective of the Freelance Hiring Platform is to provide an online system where users can manage the hiring process easily and securely. The system supports account creation, role-based access, job posting, application management, profile updates, and administration.

The project objectives include:

### 1. Creating a User-Friendly Freelance Platform

The website is designed to be simple and easy to use. Users can move between the home page, jobs page, dashboard, profile page, and admin panel using a clear navigation bar and organized layout.

### 2. Supporting Job Seekers

The job seeker can use the website to:

- Create an account and log in securely.
- View available jobs.
- Apply for jobs using name, phone number, email, CV name, CV link, and message.
- Track applied jobs and application status.
- Create and update a personal profile.
- Add bio, skills, experience, and resume information.

### **3. Supporting Employers**

The employer can use the website to:

- Create new job posts.
- View the jobs they created.
- See applicants for each job.
- Accept or reject applications.
- Send decision notifications when an application status changes.
- Update or delete their own job posts.

### **4. Supporting Admin Users**

The updated project includes an admin system. Admins can access a protected admin dashboard to view statistics about users, roles, logins, and active accounts. Super admins can also create admins, update permissions, and remove admins.

### **5. Using Modern Full-Stack Technologies**

The project uses React, Vite, JavaScript, CSS, Node.js, Express, MongoDB, Mongoose, JWT, bcryptjs, Nodemailer, and Vercel configuration. These technologies make the application dynamic, organized, secure, and deployable.

### **6. Organizing Data Using MongoDB**

MongoDB stores users, jobs, embedded job applications, admin accounts, login statistics, user profiles, and role-related information. This helps the platform display personalized dashboards and protect private data.

## **3. Technologies Used**

This project was developed using several important technologies. Each technology has a specific role in the website.

### **3.1 JavaScript**

JavaScript is the main programming language used in the project. It controls the logic and interactivity of the frontend and is also used in the backend through Node.js. It handles form submissions, routing logic, API calls, user sessions, job applications, and dashboard behavior.

### **3.2 React and Vite**

React was used to build the frontend interface as reusable components. Vite was used as the frontend development and build tool. React components include the home page, login, register, jobs page, create job page, dashboards, profile page, admin panel, navigation bar, and footer.

### **3.3 CSS**

CSS was used to style the website and improve the user experience. Separate CSS files were created for authentication, profile, jobs, dashboards, admin page, navbar, footer, home page, and global styling.

### **3.4 Node.js and Express**

Node.js and Express were used to build the backend server and REST API routes. The backend manages authentication, users, jobs, applications, admin operations, email notification logic, and database connection.

## 3.5 MongoDB and Mongoose

MongoDB is the NoSQL database used to store the website data. Mongoose was used to define schemas and interact with MongoDB in an organized way. The main models are User, Job, and Admin.

In this project, MongoDB was used to store:

- User accounts and roles.
- Hashed passwords and login activity.
- Job posts and employer information.
- Applications embedded inside jobs.
- Application status such as in-process, accepted, or rejected.
- User profile information such as bio, skills, experience, and resume.
- Admin accounts, permissions, and dashboard statistics.

## 3.6 JWT, bcrypt, Nodemailer, and Vercel

**JWT** is used to protect private routes and prove that the user is logged in. **bcryptjs** hashes passwords before saving them in the database. **Nodemailer** supports email notifications when an employer accepts or rejects an application. **Vercel** configuration is used to deploy the frontend and serverless backend API.

# 4. System Users and Roles

The system has three main user types: job seeker, employer, and admin. Each role has different permissions and features.

## 4.1 Job Seeker

A job seeker is a user looking for freelance work. The job seeker can browse available jobs, submit applications, update profile details, and view application status in the dashboard.

## **Job Seeker Features**

- Create an account and log in.
- View available jobs.
- Apply for jobs only once.
- Submit name, phone number, email, CV name, CV link, and message.
- Open a dashboard to see applied jobs and application status.
- Open and update a profile page.
- Add bio, skills, experience, and resume information.

## **4.2 Employer**

An employer is a user who wants to hire freelancers. The employer can create job posts, view applicants, update job data, delete jobs, and accept or reject applications.

### **Employer Features**

- Create an account and log in.
- Add new job posts.
- View jobs created by the employer.
- Review applicants for each job.
- Accept or reject applicants.
- Trigger email notifications after decision changes.
- Update or delete only their own jobs.

## **4.3 Admin and Super Admin**

The updated project includes an admin role. Admins can access a protected admin dashboard. A super admin has higher permissions and can create admins, update permissions, and remove admins.

## **5. Main Features of the Website**

The Freelance Hiring Platform includes several important features that support job seekers, employers, and administrators.

### **5.1 Home Page and Layout**

The home page introduces the platform and uses a clean layout supported by the Navbar and Footer components. The App component defines the main routes for all pages.

### **5.2 Login and Authentication**

Users log in using email and password. Authentication is handled using JWT tokens. Tokens are stored on the frontend and sent in the x-auth-token header to access protected routes.

## **5.3 Create Job Page**

The create job page allows employers to add new job opportunities. Job details include title, description, company, location, salary range, job type, experience level, and skills.

## **5.4 Job Browsing and Applications**

The jobs page allows users to browse available jobs. Job seekers can apply to a job by submitting their contact information and CV details. The backend prevents duplicate applications to the same job.

## **5.5 Job Seeker Dashboard**

The job seeker dashboard displays jobs that the current user applied for. It also shows whether each application is still in process, accepted, or rejected.

## **5.6 Job Seeker Profile Page**

The profile page allows users to update their name, bio, skills, experience, and resume information. Profile information is stored inside the User model.

## **5.7 Employer Dashboard**

The employer dashboard displays jobs created by the employer and the list of applicants for each job. This helps the employer manage the hiring process from one organized page.

## **5.8 Application Status and Email Notifications**

Employers can change an application status to accepted or rejected. When the status changes, the backend can send an email notification to the applicant using Nodemailer.

## **5.9 Admin Panel**

The admin panel displays statistics such as total users, active users, job seekers, employers, users with logins, and total admins. It also supports admin and permission management for super admins.



## 6. Database Design

MongoDB was used as the database for this project. MongoDB stores data in collections and documents. Mongoose schemas were used to structure the main data models.

### 6.1 Main Database Collections

#### Users Collection

This collection stores user account information, login tracking, role information, and profile data.

Example fields:

| Field      | Description                         |
|------------|-------------------------------------|
| name       | User full name                      |
| email      | Unique user email                   |
| password   | Hashed password                     |
| role       | jobseeker or employer               |
| profile    | Bio, skills, experience, and resume |
| lastLogin  | Last login date                     |
| loginCount | Number of user logins               |
| isActive   | Active or inactive account status   |

#### Jobs Collection

This collection stores jobs created by employers. Applications are stored inside the related job document.

Example fields:

| Field           | Description                                  |
|-----------------|----------------------------------------------|
| title           | Job title                                    |
| description     | Job description                              |
| company         | Company name                                 |
| location        | Job location                                 |
| salary          | Minimum and maximum salary                   |
| jobType         | Full-time, part-time, contract, or temporary |
| experienceLevel | Entry, intermediate, or senior               |
| skills          | Required skills                              |
| postedBy        | Employer who created the job                 |

## Embedded Applications

Applications are embedded inside the Jobs collection. This makes it easy for employers to view applicants for a specific job.

Example fields:

| Field     | Description                       |
|-----------|-----------------------------------|
| userId    | The job seeker who applied        |
| name      | Applicant name                    |
| phone     | Applicant phone number            |
| email     | Applicant email                   |
| cvName    | CV file name or title             |
| cvUrl     | CV link                           |
| message   | Application message               |
| status    | in-process, accepted, or rejected |
| appliedAt | Date of application               |

## Admins Collection

This collection stores admin users and their permissions.

| Field          | Description           |
|----------------|-----------------------|
| userId         | Related user account  |
| name           | Admin name            |
| email          | Admin email           |
| role           | admin or superadmin   |
| permissions    | Allowed admin actions |
| status         | active or inactive    |
| lastAdminLogin | Last admin login date |

## 6.2 Importance of the Database

The database is important because it keeps all project data saved and organized. It allows different users to see different information based on their role. For example, job seekers see their own applications, employers see their own posted jobs, and admins see platform statistics.

## 7. Implementation Overview

The project was implemented as a full-stack web application. The frontend is divided into React components, while the backend is divided into routes, models, middleware, utilities, and deployment entry files.

### 7.1 Job Seeker Flow

The job seeker flow can be described as follows:

- The job seeker opens the website.
- The job seeker creates an account or logs in.
- The frontend stores the JWT token and user data.
- The job seeker browses available jobs.
- The job seeker submits an application.
- The applied job appears in the dashboard.
- The job seeker checks whether the application is in-process, accepted, or rejected.
- The job seeker updates profile information.

### 7.2 Employer Flow

The employer flow can be described as follows:

- The employer creates an account or logs in.
- The employer creates a job post.
- The created jobs appear in the employer dashboard.
- The employer views applicants for each job.
- The employer accepts or rejects applications.
- An email notification can be sent to the applicant after the decision.

### 7.3 Admin Flow

The admin logs in and opens the admin panel. The backend checks the JWT token and verifies that the user exists in the Admin collection. Admins can view dashboard statistics and user data, while super admins can manage admin permissions.

### 7.4 Frontend Implementation

The frontend uses React Router to manage pages such as Home, Login, Register, Jobs, Dashboard, Profile, Admin, and Create Job. The AuthContext stores user session data and provides login, register, logout, and update user functions to the whole application.

## 7.5 Backend, API, and Deployment

The backend uses Express routes for authentication, jobs, users, and admin functions. The auth middleware checks the JWT token before protected operations. The server connects to MongoDB and exports the Express app so it can be used by the Vercel serverless API entry file.

Vercel deployment is configured using `vercel.json`. Requests beginning with `/api` are routed to `api/index.js`, while the React frontend is built as a static site from the frontend folder.

## 8. Testing and Validation

Testing was an important step to make sure the website works correctly for job seekers, employers, and admins.

### 8.1 Job Seeker Testing

| Test Case                    | Expected Result                               |
|------------------------------|-----------------------------------------------|
| Job seeker registers         | Account is created and token is returned      |
| Job seeker logs in           | User enters the website successfully          |
| Job seeker applies for a job | Application is saved inside the job           |
| Job seeker opens dashboard   | Applied jobs and status are displayed         |
| Job seeker updates profile   | Bio, skills, resume, and experience are saved |

### 8.2 Employer Testing

| Test Case                          | Expected Result                                   |
|------------------------------------|---------------------------------------------------|
| Employer logs in                   | Employer dashboard opens                          |
| Employer creates a job             | Job is added successfully                         |
| Employer views applicants          | Applicants for each job are shown                 |
| Employer accepts/rejects applicant | Application status is updated                     |
| Email notification runs            | Applicant receives or backend logs decision email |

### 8.3 Admin and Deployment Testing

| Test Case                   | Expected Result                           |
|-----------------------------|-------------------------------------------|
| Admin opens dashboard       | Statistics are displayed                  |
| Non-admin opens admin panel | Access is denied                          |
| Super admin creates admin   | New admin is saved                        |
| API request uses token      | Protected backend route allows valid user |
| Vercel route opens frontend | Application loads and API routes work     |

## 9. Conclusion

The project was built using JavaScript, React, Vite, CSS, Node.js, Express, MongoDB, Mongoose, JWT, bcryptjs, Nodemailer, and Vercel deployment configuration. React helped create a dynamic frontend, CSS improved the design, Express handled APIs, MongoDB stored the data, and JWT protected private actions.

Overall, the project successfully provides a stronger freelance hiring system than the first version because it now includes backend security, application status management, admin control, email notification logic, and deployment support.

## 10. Future Enhancements

In the future, the project can be improved by adding more advanced features such as:

- Real-time chat between employers and job seekers.
- Advanced job search and filtering.
- Real CV file upload storage instead of only CV name or link.
- More detailed notification settings.
- Rating and review system.
- Payment system for freelance work.
- More advanced admin analytics.
- Improved responsive design for mobile devices.
- Password reset by email.
- More security features such as refresh tokens and stronger authorization controls.

## Final Summary

This project helped us understand how to build a real full-stack web application using modern technologies. It also gave us experience in frontend development, backend APIs, database design, authentication, user roles, dashboards, admin management, email utilities, and deployment preparation.

The Freelance Hiring Platform can be considered a strong base for a complete online freelance system because it includes the main features needed for users to register, log in, post jobs, apply for jobs, track decisions, manage profiles, and control platform data through an admin dashboard.